অধ্যায় ৭

Objects: Properties, Methods, Destructuring এবং Real-World Data Modeling

শেখার লক্ষ্য

এই অধ্যায় শেষে পাঠক JavaScript Object তৈরি, পড়া, পরিবর্তন, Method লেখা, Reference বোঝা, Destructuring ও Spread ব্যবহার, Object Validation এবং Student Management Application তৈরি করতে পারবেন। আগের অধ্যায়ে Array-তে অনেক Record রাখা শিখেছি; এখানে প্রতিটি Record-এর অর্থপূর্ণ Structure তৈরি করব।

৭.১ Object কেন প্রয়োজন?

ধরুন একটি শিক্ষার্থীর নাম, বয়স ও নম্বর আছে। তিনটি পৃথক Variable-এ রাখা যায়, কিন্তু একাধিক শিক্ষার্থী এলে কোন নামের সঙ্গে কোন নম্বর যুক্ত তা বোঝা কঠিন হয়। Object একই Entity-এর সম্পর্কিত তথ্যকে Key–Value Pair-এ সংগঠিত করে।

```javascript
const student = {
  id: 'ST-101',
  name: 'Amina Rahman',
  age: 19,
  marks: 86,
  isActive: true
};
console.log(student.name);       // Amina Rahman
console.log(student['marks']);   // 86
```

`student` একটি Object Reference ধরে। `id`, `name`, `age`, `marks` হলো Properties। Array সাধারণত Ordered Collection, আর Object সাধারণত Named Fields-এর জন্য ব্যবহার হয়। তবে JavaScript-এ Array-ও এক ধরনের Object; প্রয়োগের উদ্দেশ্য আলাদা।

৭.২ Object Literal, Constructor এবং `Object.create()`

```javascript
const book = { title: 'JavaScript Mastery', pages: 300 };
const settings = new Object();
settings.theme = 'light';
const dictionary = Object.create(null);
dictionary['__proto__'] = 'safe as an ordinary key';
```

অধিকাংশ Application Data-এর জন্য Object Literal সহজ। `Object.create(null)`-এর Prototype নেই, তাই Dictionary-সদৃশ কাজে বিশেষ সুবিধা দিতে পারে; তবে এতে `toString()`-এর মতো স্বাভাবিক Inherited Method থাকে না।

৭.৩ Dot Notation বনাম Bracket Notation

```javascript
const profile = { firstName: 'Amina', 'home-city': 'Dhaka' };
console.log(profile.firstName);
console.log(profile['home-city']);
const field = 'firstName';
console.log(profile[field]);
```

Key যদি Dynamic হয়, Space/Hyphen থাকে, অথবা Runtime-এ নির্ধারিত হয়, Bracket Notation ব্যবহার করতে হবে। `profile.field` এবং `profile[field]` এক নয়: প্রথমটি আক্ষরিক `field` Property খোঁজে, দ্বিতীয়টি Variable-এর Value ব্যবহার করে।

৭.৪ Property Read, Update, Delete এবং Existence

```javascript
const user = { name: 'Rahim', city: 'Dhaka' };
user.city = 'Chattogram';
user.email = 'rahim@example.com';
delete user.email;
console.log(Object.hasOwn(user, 'name')); // true
console.log('toString' in user);          // true: inherited
```

`Object.hasOwn()` শুধু Object-এর নিজস্ব Property পরীক্ষা করে; `in` Operator Prototype Chain-এও খোঁজে। Property অনুপস্থিত এবং Property-এর Value `undefined`—এ দুটো আলাদা ঘটনা।

৭.৫ Methods এবং `this`

```javascript
const account = {
  owner: 'Amina',
  balance: 500,
  deposit(amount) {
    if (!Number.isFinite(amount) || amount <= 0) throw new Error('Invalid amount');
    this.balance += amount;
    return this.balance;
  }
};
console.log(account.deposit(200)); // 700
```

Method-এর `this` সাধারণত Call-site-এর ওপর নির্ভর করে। `account.deposit()`-এ `this` হলো `account`; Method আলাদা Variable-এ রেখে Call করলে `this` হারাতে পারে। Arrow Function-এর নিজস্ব `this` নেই, তাই Object Method-এ প্রয়োজন বুঝে ব্যবহার করুন।

৭.৬ Nested Objects এবং Optional Chaining

```javascript
const traveler = {
  name: 'Nadia',
  contact: { phone: '01700000000', address: { city: 'Dhaka' } }
};
console.log(traveler.contact?.address?.city); // Dhaka
console.log(traveler.passport?.number ?? 'Not provided');
```

`?.` অনুপস্থিত Nested Data Access-এ TypeError এড়াতে সাহায্য করে; `??` শুধু `null` বা `undefined` হলে Fallback দেয়। তবে এটি Schema Validation-এর বিকল্প নয়।

৭.৭ Object Destructuring এবং Defaults

```javascript
const student = { id: 'ST-101', name: 'Amina', marks: 86 };
const { name, marks: score, city = 'Unknown' } = student;
console.log(name, score, city);

function printStudent({ name, marks = 0 }) {
  return `${name}: ${marks}`;
}
```

`marks: score` মানে `marks` Property-এর Value নতুন `score` Variable-এ যাবে। Default Value কেবল `undefined` হলে প্রযোজ্য; `null` হলে নয়। Function Parameter-এ Destructuring API-কে স্পষ্ট করে।

৭.৮ Spread, Rest এবং Shallow Copy

```javascript
const original = { id: 'ST-101', name: 'Amina', address: { city: 'Dhaka' } };
const updated = { ...original, name: 'Amina Rahman' };
const { id, ...publicInfo } = updated;
console.log(publicInfo.name);
console.log(updated.address === original.address); // true
```

Spread Copy **Shallow**: প্রথম স্তরের Property Copy হয়, কিন্তু Nested Object-এর Reference ভাগাভাগি হতে পারে। Nested Data বদলাতে হলে প্রয়োজনমতো Nested Copy তৈরি করুন। `structuredClone()` অনেক Built-in Data Type Deep-copy করতে পারে, কিন্তু Function, DOM Node ইত্যাদি Clone করতে পারে না।

৭.৯ Reference, Equality ও Mutation

```javascript
const a = { marks: 80 };
const b = a;
const c = { marks: 80 };
console.log(a === b); // true
console.log(a === c); // false
b.marks = 90;
console.log(a.marks); // 90
```

Object Equality সাধারণত Reference Identity পরীক্ষা করে, Content Equality নয়। তাই State Management-এ পুরোনো Object পরিবর্তন না করে নতুন Object Return করা সুবিধাজনক।

৭.১০ Object Utility Methods

```javascript
const scores = { math: 80, english: 75, science: 90 };
console.log(Object.keys(scores));
console.log(Object.values(scores));
console.log(Object.entries(scores));
const doubled = Object.fromEntries(
  Object.entries(scores).map(([subject, mark]) => [subject, mark * 2])
);
console.log(doubled);
```

`Object.keys()` / `values()` / `entries()` Enumerable Own String-Keyed Properties নিয়ে কাজ করে। `Object.freeze()`-ও Shallow; Nested Objects স্বয়ংক্রিয়ভাবে Deep-Freeze হয় না।

৭.১১ JSON এবং Browser Storage

```javascript
const record = { id: 'ST-101', name: 'Amina', marks: 86 };
const serialized = JSON.stringify(record);
const restored = JSON.parse(serialized);
```

JSON Data Exchange-এর Format, JavaScript Object-এর পূর্ণ বিকল্প নয়। JSON-এ Function, `undefined`, Circular Reference এবং `BigInt`-এর মতো Value-এর সীমাবদ্ধতা আছে। Untrusted JSON Parse করার পর Schema Validation জরুরি। `localStorage` String রাখে, তাই Serialize/Parse করতে হয়। Browser Storage-এ Password, Secret Token বা Sensitive Student Data রাখা উচিত নয়।

৭.১২ Real-World Data Modeling: Student Schema

এই অধ্যায়ের Project-এ একটি Student Record-এর Schema:

Field	Type	Validation
`id`	String	২–২০ Character, Unique
`name`	String	২–৮০ Character
`age`	Integer	৫–১০০
`marks`	Number	০–১০০

একজন শিক্ষার্থীকে এভাবে Represent করা হবে:

```javascript
const student = { id: 'ST-101', name: 'Amina Rahman', age: 19, marks: 86 };
```

Project-এর Business Logic-এ `add`, `update`, `remove`, `search`, `gradeFor`, `summary` Function থাকবে। প্রতিটি Function Data Return করবে; Browser DOM Logic আলাদা File-এ থাকবে। এতে Testing ও Maintenance সহজ হয়।

৭.১৩ Practical Project — Student Management Dashboard

Feature List

1. Student Add, Edit এবং Delete
2. Duplicate Student ID প্রতিরোধ
3. Name ও ID দিয়ে Search
4. Grade Calculation (A+, A, A−, B, C, D, F)
5. Total Students, Average Marks, Passed Students, Topper
6. `localStorage` Persistence এবং Invalid Stored Record Filtering
7. DOM-এ User Input Render করার সময় `textContent` ব্যবহার

File Structure

```
source-code/chapter-07/
├── index.html
├── style.css
├── students.js
├── app.js
└── students.test.js
```

Runnable Code: বইয়ের Companion Source Code-এর `source-code/chapter-07/` Folder-এ প্রতিটি সম্পূর্ণ File সংযুক্ত আছে।

Business Logic-এর গুরুত্বপূর্ণ অংশ

```javascript
function add(students, input) {
  return [...students, validateStudent(input, students)];
}
function update(students, id, patch) {
  const old = students.find(s => s.id === id);
  if (!old) throw new Error('Student not found');
  const next = validateStudent({ ...old, ...patch }, students, id);
  return students.map(s => s.id === id ? next : s);
}
```

এখানে `add` এবং `update` পুরোনো Array পরিবর্তন করে না। একই সঙ্গে Duplicate ID, Invalid Marks ও Age Reject করা হয়।

Run Instructions

1. `source-code/chapter-07/index.html` Browser-এ Open করুন।
2. Student ID, Name, Age, Marks দিয়ে Save করুন।
3. Search, Edit, Delete এবং Refresh করে Persistence পরীক্ষা করুন।
4. Node.js Installed থাকলে Terminal-এ `node students.test.js` চালান।

Manual Test Checklist

- ☐ Add করার পর Table-এ Student দেখা যায়
- ☐ Duplicate ID দিলে Error দেখায়
- ☐ Edit করলে Student-এর Data Update হয়
- ☐ Delete করলে Confirmation চায়
- ☐ Search Uppercase/Lowercase নির্বিশেষে কাজ করে
- ☐ Page Refresh-এর পর Valid Data থাকে
- ☐ Keyboard দিয়ে Form ব্যবহার করা যায়

৭.১৪ Common Mistakes

Mistake 1: Object Copy মানেই Deep Copy ভাবা। Spread-এর Nested Reference একই থাকতে পারে।

Mistake 2: `for...in` দিয়ে Blind Iteration। এটি Inherited Enumerable Properties-ও দেখতে পারে। Own Data-এর জন্য `Object.keys()` / `entries()` ব্যবহার করুন।

Mistake 3: Untrusted Data সরাসরি `innerHTML`-এ বসানো। XSS Risk তৈরি হতে পারে; Plain Text-এর জন্য `textContent` ব্যবহার করুন।

Mistake 4: `Number('')` যে `0` হয় তা ভুলে যাওয়া। Form Validation-এ Empty Input আলাদাভাবে Reject করতে হবে।

Mistake 5: Frontend Validation-কে Security Boundary ভাবা। Real Server Application-এ Backend Validation ও Authorization প্রয়োজন।

৭.১৫ Exercises

1. একটি `book` Object তৈরি করুন, যার `title`, `author`, `price` আছে। Price ১০% কমিয়ে নতুন Object তৈরি করুন।
2. `getFullName({ firstName, lastName })` Function লিখুন।
3. `Object.entries()` ব্যবহার করে Subject-wise Marks-এর Total বের করুন।
4. `address?.city ?? 'Unknown'` Expression কীভাবে কাজ করে ব্যাখ্যা করুন।
5. একটি Array of Student Objects থেকে ৮০ বা বেশি Marks পাওয়া Student Filter করুন।
6. `Object.hasOwn()` এবং `in`-এর পার্থক্য Code দিয়ে দেখান।
7. Nested Object-এর Shallow Copy Problem দেখিয়ে Corrected Copy তৈরি করুন।
8. Student Dashboard-এ Sort by Marks Feature যুক্ত করুন।
9. `validateStudent()`-এ Duplicate ID Checking কেন দরকার ব্যাখ্যা করুন।
10. Student Data Export-এর জন্য `JSON.stringify()` ব্যবহার করুন এবং Import-এর সময় Validation যুক্ত করুন।

৭.১৬ Exercise Solutions

JAVASCRIPT

```
// 1
const book = { title: 'JS', author: 'A', price: 500 };
const discounted = { ...book, price: book.price * 0.9 };

// 2
const getFullName = ({ firstName, lastName }) => `${firstName} ${lastName}`.trim();

// 3
const marks = { math: 80, english: 70, science: 90 };
const total = Object.entries(marks).reduce((sum, [, value]) => sum + value, 0);

// 4: address null/undefined হলে city access বন্ধ হয়; result null/undefined হলে Unknown
const profile = {};
const city = profile.address?.city ?? 'Unknown';

// 5
const topStudents = [{name:'A',marks:82},{name:'B',marks:65}].filter(s => s.marks >= 80);

// 6
const obj = Object.create({ inherited: true });
obj.own = true;
console.log('inherited' in obj);          // true
console.log(Object.hasOwn(obj,'inherited')); // false

// 7
const oldRecord = { name: 'A', address: { city: 'Dhaka' } };
const newRecord = { ...oldRecord, address: { ...oldRecord.address, city: 'Khulna' } };
```

```
// 8: data sorting; UI-তে sort select-এর change handler-এ ব্যবহার করুন
const sortByMarks = students => [...students].sort((a, b) => b.marks - a.marks);

// 9: unique ID ছাড়া update/delete ভুল শিক্ষার্থীকে প্রভাবিত করতে পারে
// 10: parse-এর পর প্রত্যেক Record validate করে গ্রহণ করুন
const exportJSON = students => JSON.stringify(students, null, 2);
const importJSON = text => {
  const parsed = JSON.parse(text);
  if (!Array.isArray(parsed)) throw new Error('Expected array');
  return parsed.reduce((acc, record) => StudentStore.add(acc, record), []);
};
```

৭.১৭ Knowledge Check — ১০টি MCQ

1. Object Property Access-এর জন্য কোনটি সঠিক? A. `obj.name` B. `obj->name` C. `obj::name` D. `obj#name`
2. Dynamic Key-এর জন্য কোন Syntax? A. `obj.key` B. `obj[key]` C. `obj(key)` D. `obj{key}`
3. Spread দিয়ে Nested Object Copy করলে কী হয়? A. Deep Copy B. Shallow Copy C. JSON Copy D. Freeze
4. Own Property পরীক্ষা করে কোনটি? A. `Object.hasOwn()` B. `in` only C. `typeof` D. `Array.isArray()`
5. `const a={}; const b=a; a===b` ? A. false B. true C. null D. Error
6. `Object.entries()` কী Return করে? A. String B. Key–Value Pair Array C. Set D. Number
7. `user.profile?.name`-এ `profile` অনুপস্থিত হলে Result? A. Error B. 0 C. undefined D. false
8. JSON Data Parse করার Method? A. `JSON.read` B. `JSON.parse` C. `JSON.open` D. `JSON.decodeObject`
9. User Text DOM-এ নিরাপদ Plain Text হিসেবে বসাতে কোনটি? A. `innerHTML` B. `eval` C. `textContent` D. `document.write`
10. Student Marks 0–100 নিশ্চিত করার সঠিক পদ্ধতি? A. Only HTML `min/max` B. Explicit Validation C. `parseInt` only D. Ignore

Answer Key এবং Explanation

Q	Answer	কারণ
1	A	Dot Notation সরাসরি Named Property Access করে
2	B	Bracket Notation Variable-এর Value ব্যবহার করে
3	B	Spread Nested Reference Deep Copy করে না
4	A	Own Property-ই পরীক্ষা করে
5	B	দুই Variable একই Object Reference ধরে
6	B	`[key, value]` Pair-এর Array দেয়
7	C	Optional Chaining Access বন্ধ করে `undefined` দেয়
8	B	JSON Text-কে JavaScript Value-তে রূপান্তর করে
9	C	HTML হিসেবে Interpret না করে Text বসায়

Q	Answer	কারণ
10	B	Explicit Validation দরকার; HTML Constraint Bypass হতে পারে

৭.১৮ Chapter Summary এবং পরবর্তী অধ্যায়

Objects আমাদের Named Data, Methods, Nested Records এবং Real-World Entity Modeling-এর ভিত্তি। Destructuring, Spread, Optional Chaining ও Object Utilities দিয়ে Maintainable Code লেখা যায়। Student Management Project-এ এই ধারণাগুলো একসঙ্গে ব্যবহার করা হয়েছে। পরবর্তী অধ্যায় **Chapter 8 — Strings এবং Modern Text Processing**-এ Text Transformation, Unicode, Search এবং Input Processing শিখব।